

Guía de ejercicios - El framework Spring (II)



¡Hola! Te damos la bienvenida a esta nueva guía de estudio.

¿En qué consiste esta guía?

La siguiente guía de estudio tiene como objetivo practicar y ejercitar los contenidos que hemos visto en clase. Para ello hemos dispuesto de ejercicios que te permitan guiar tu ejercitación y también hacerlo de manera autónoma.

Te invitamos a desarrollar este material para poder practicar y seguir aprendiendo.

¡Vamos con todo!



Tabla de contenidos

Actividad guiada: Desarrollo de un proyecto Spring con JUnit.	3
Creación de Proyecto	3
Creación del modelo y servicio.....	7
Clase ApplicationTests	9
Probando el Proyecto	9
Resultado de la ejecución del Proyecto	10
¡Manos a la obra! - Clases Clientes	11
Solución del Proyecto.....	12
Preguntas de proceso	14
Referencias bibliográficas	14



¡Comencemos!



Actividad guiada: Desarrollo de un proyecto Spring con JUnit.

En este ejercicio vamos a desarrollar un nuevo proyecto **Java Web con Spring Boot**, en donde vamos a crear test, para validar algunos métodos en un servicio.

Para lograrlo, necesitarás aplicar todo lo aprendido en las clases anteriores de esta unidad, (te aconsejamos mantener a mano dicho material en caso de cualquier duda).

Aplicando los conceptos y herramientas aprendidas hasta ahora, generamos lo siguiente:

1. Un proyecto Java Web Spring Boot llamado “EjemploSpringJUnit”.
2. Al crear el proyecto, verificamos que haya creado el método principal “main”, el cual es el encargado de la ejecución de nuestro proyecto.
3. Creación de paquetes y clases Java necesarias.
4. Creación de métodos test.
5. Generación de respuesta en la terminal de consola y JUnit.

¡Vamos con todo!

Creación de Proyecto

Para crear el proyecto desde haz clic con el botón secundario del mouse, selecciona “**New**”, luego selecciona “**Other**”.

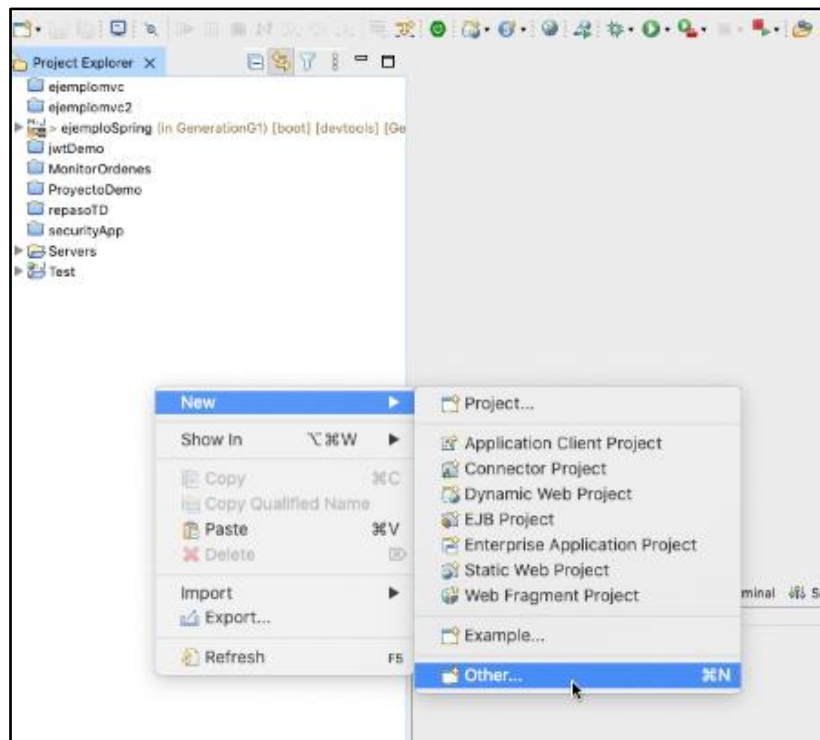


Imagen 1. Creación de Proyecto

Fuente: Desafío Latam

Se desplegará una pantalla, en la cual en **“Wizard”** escribe **“spring”** y se filtraran automáticamente y podremos seleccionar la opción **“Spring Starter Project”**.

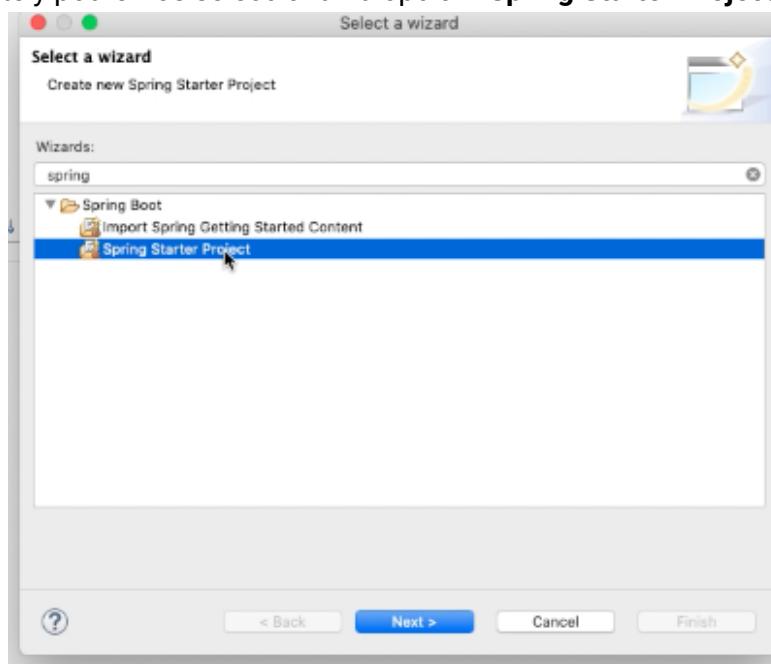


Imagen 2. Creación de Proyecto

Fuente: Desafío Latam

A continuación, se debe de configurar:

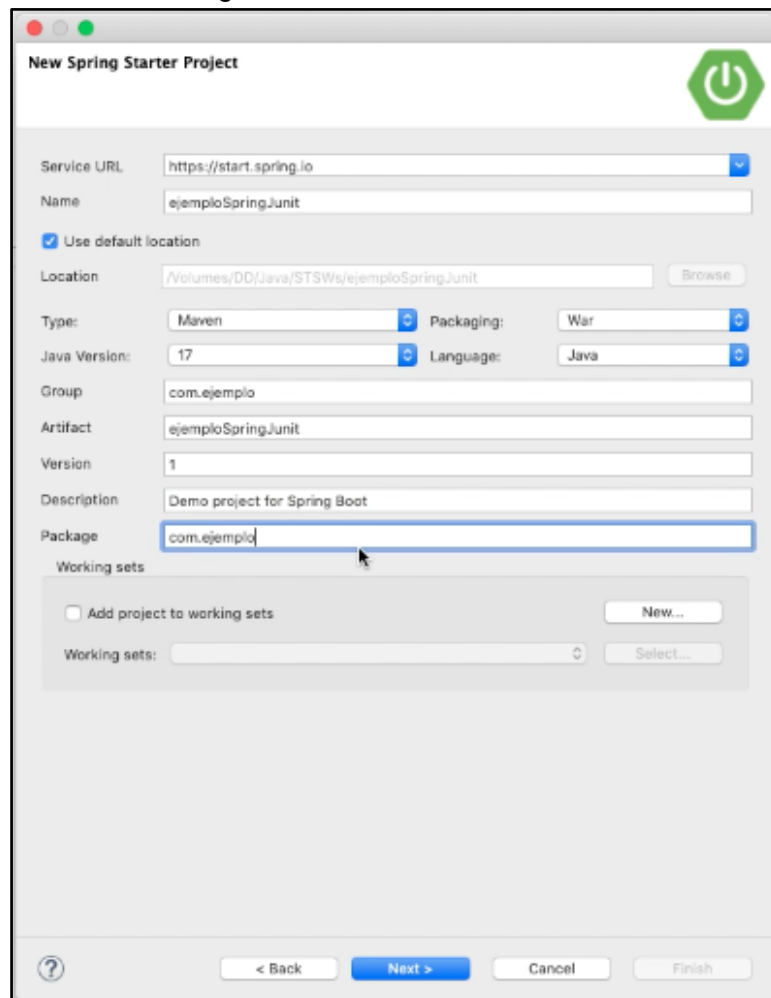


Imagen 3. Creación de Arquitectura Proyecto
Fuente: Desafío Latam

A continuación agregaremos las dependencias base de nuestro proyecto “Spring Web” y presionamos “Finish”.

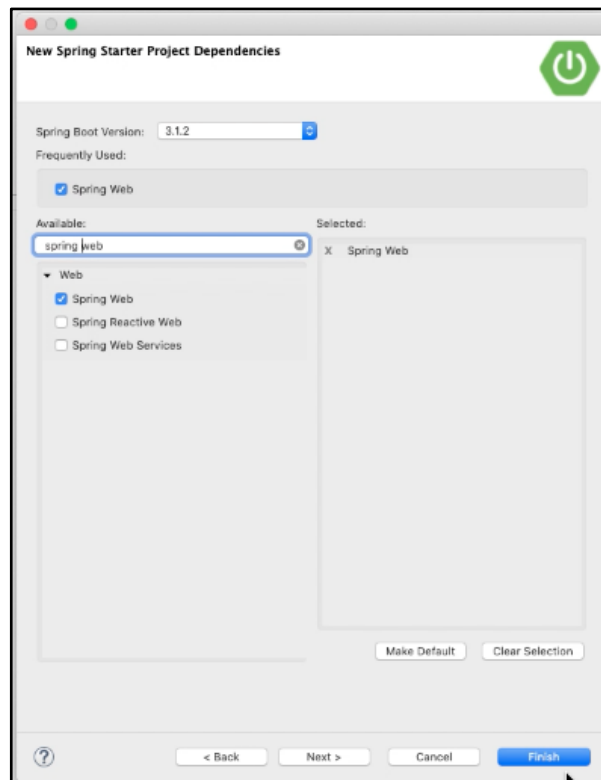


Imagen 4. Selección de dependencias.

Fuente: Desafío Latam



¡Lo lograste!

Creaste tu **proyecto** por medio del IDE **Spring Tool Suite**.

Creación del modelo y servicio.

Para realizar las pruebas con Junit, crearemos un servicio que retorne un listado de usuarios, para lo cual deberemos crear un modelo llamado Usuario y la clase servicio con la anotación @Service, la cual será la encargada de retornar desde un método el listado.

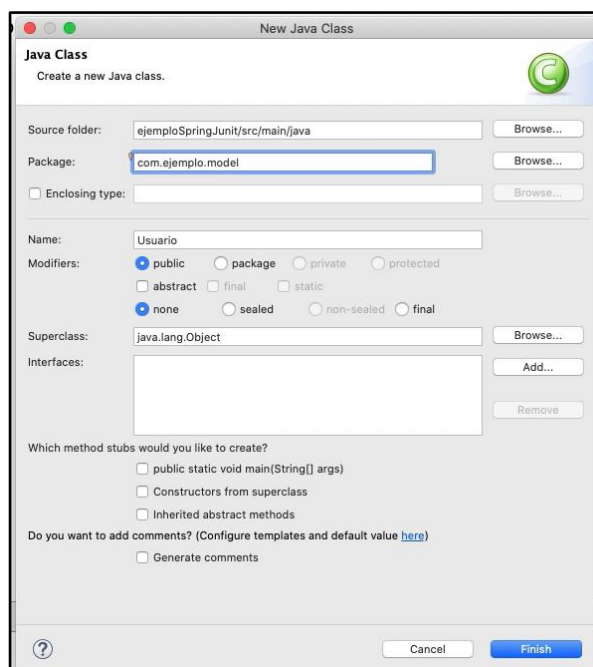


Imagen 5. Creación del modelo Usuario en su respectivo package.

Fuente: Desafío Latam

Agregamos algunos atributos al modelo, con su respectivo constructor y sus setter y getters.

```
1 package com.ejemplo.model;
2
3 public class Usuario {
4
5     private String nombre;
6     private String email;
7     private int edad;
8
9     public Usuario(String nombre, String email, int edad) {
10         super();
11         this.nombre = nombre;
12         this.email = email;
13         this.edad = edad;
14     }
15
16     public String getNombre() {
17         return nombre;
18     }
19
20     public void setNombre(String nombre) {
21         this.nombre = nombre;
22     }
23
24     public String getEmail() {
25         return email;
26     }
27
28     public void setEmail(String email) {
29         this.email = email;
30     }
31
32     public int getEdad() {
33         return edad;
34     }
35
36     public void setEdad(int edad) {
37         this.edad = edad;
38     }
39 }
40
```

Imagen 6. Creación atributos, constructor, setter y getters.

Fuente: Desafío Latam

Creamos el servicio con el método que retornará la lista, también en su respectivo package.

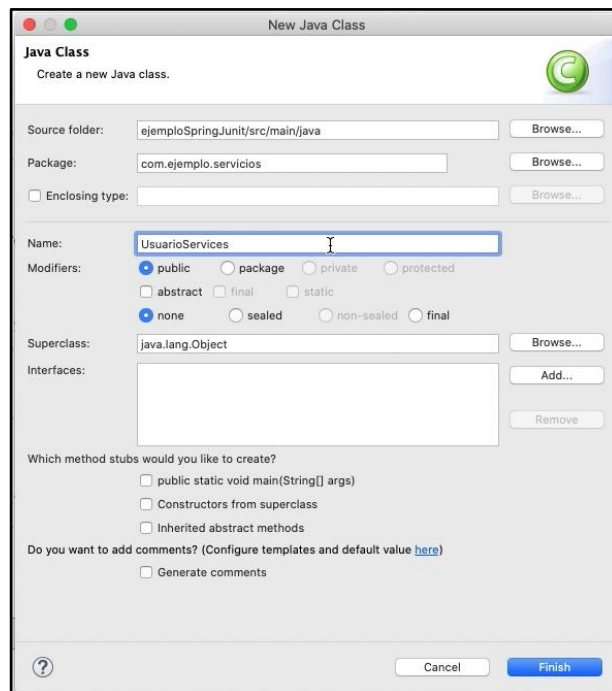


Imagen 7. Creación clase UsuarioService.
Fuente: Desafío Latam

Agregamos la anotación `@Service` sobre la clase, luego creamos el método `buscarUsuarios()` encargado de retornar la lista de usuarios. generamos unas instancias de la clase `Usuario`, invocando al constructor con parámetros, para luego agregarlos a la lista a retornar.

```
1 package com.ejemplo.servicios;
2
3 import com.ejemplo.model.Usuario;
4
5
6
7
8 @Service
9 public class UsuarioServices {
10
11     public List<Usuario> buscarUsuarios(){
12         Usuario user1 = new Usuario("Pedro", "p@gmail.com", 30);
13         Usuario user2 = new Usuario("Juan", "j@hotmail.com", 25);
14
15         List<Usuario> lista = Arrays.asList(user1, user2);
16         return lista;
17     }
18 }
19
```

Imagen 7. Creación método que retorna lista.
Fuente: Desafío Latam

Clase ApplicationTests

En el apartado src/test/java, dentro del package com.ejemplo encontraremos la clase “EjemploSpringJUnitApplicationTests”, en la cual escribiremos el test que validara que el método retorna una lista de más de un usuario.

Primero inyectamos con @Autowired el servicio creado:

```
@Autowired
UsuarioServices userservices;
```

Y luego creamos la prueba unitaria del método:

```
@Test
public void test() {
    assertEquals(2, userservices.buscarUsuarios().size());
}
```

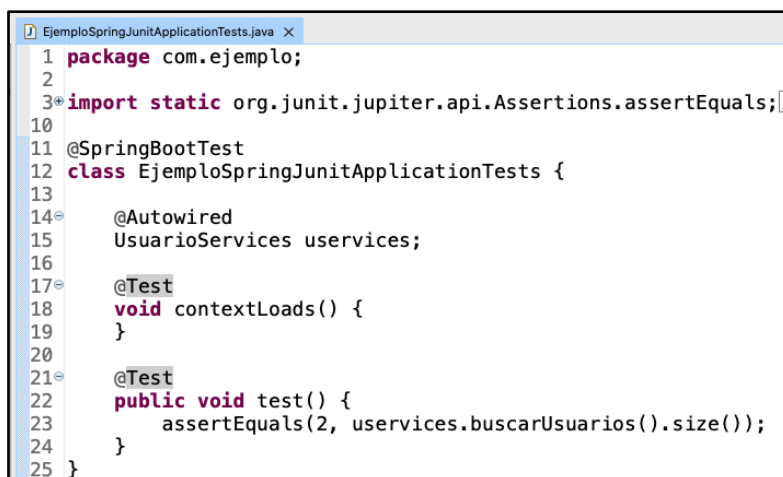


Imagen 8. Resultado final de la Clase ApplicationTests.

Fuente: Desafío Latam

Probando el Proyecto

Ya realizados todos los pasos anteriores, podemos probar el método de nuestro proyecto. Sobre la clase “**EjemploSpringJUnitApplicationTests**” damos clic con el botón derecho del mouse e irnos a “**Run As**”, y seleccionar “**JUnit Test**”:

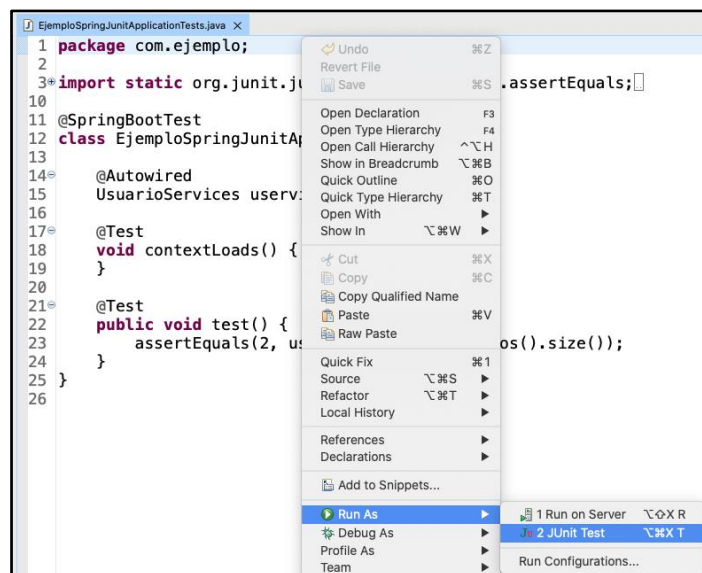


Imagen 9. Ejecución Test en Proyecto Spring Boot
Fuente: Desafío Latam

Resultado de la ejecución del Proyecto

Con lo anterior, ya podemos ver el resultado correcto de la prueba en el proyecto Java Web Spring Boot, tal como se muestra en la siguiente captura de pantalla..

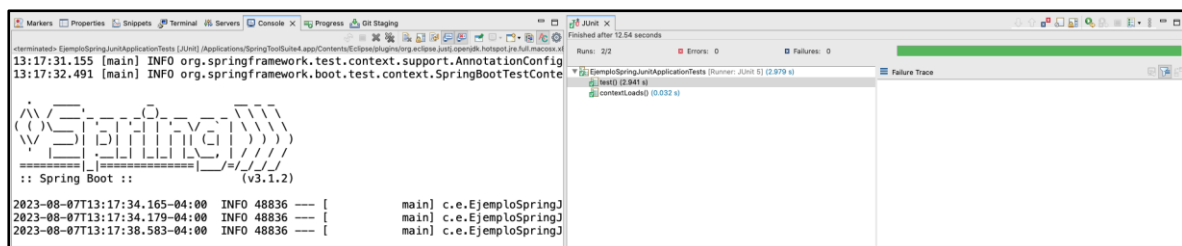


Imagen 10. Test en Proyecto Spring Boot ejecutado.
Fuente: Desafío Latam



¡Lo lograste!

Ejecutaste tu **Test** en un proyecto Spring Boot utilizando JUnit.



¡Manos a la obra! - Clases Clientes

Continuando con el mismo proyecto el Product Owner a solicitado realizar los métodos básicos de un crud (crear, leer, actualizar y eliminar) para los clientes, por ahora solo se necesita simular los métodos en el servicio para posteriormente proceder al desarrollo TDD (Test Driven Development). Para esto se solicita:

1. Método de creación de un cliente.
2. Método de actualización de un cliente.
3. Método de eliminación de un cliente (retornar mensaje "Eliminación realizada correctamente").
4. Selección de un cliente.
5. Selección de listado de clientes.

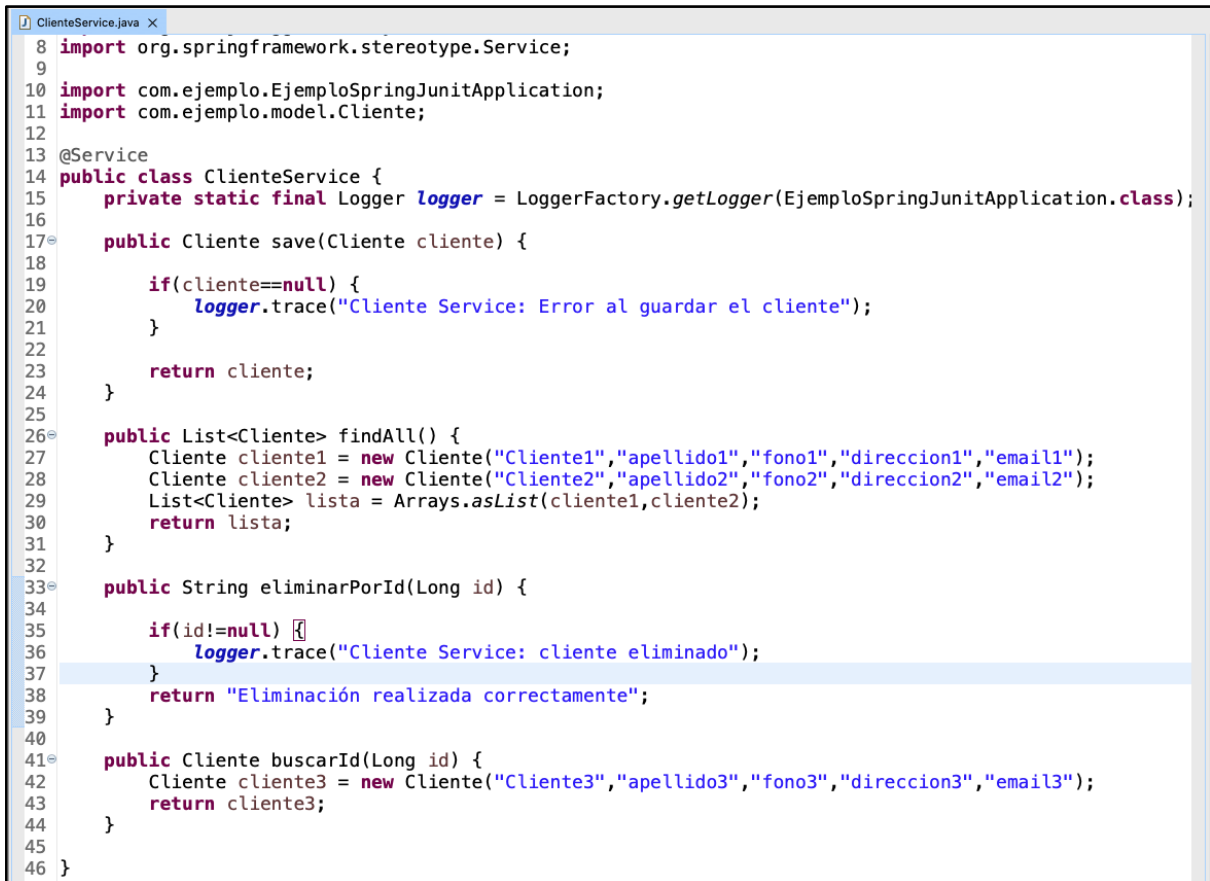
Considerar los atributos del Cliente:

- Nombre
- Apellido
- Fono
- Dirección
- Email

Además, se deben ejecutar todas las pruebas correctamente.

Solución del Proyecto

Con lo anterior, ya podemos ver el resultado correcto de la prueba en el proyecto Java Web Spring Boot, tal como se muestra en la siguiente captura de pantalla:



```
1  ClienteService.java x
2
3  import org.springframework.stereotype.Service;
4
5
6  import com.ejemplo.EjemploSpringJUnitApplication;
7  import com.ejemplo.model.Cliente;
8
9  @Service
10 public class ClienteService {
11     private static final Logger logger = LoggerFactory.getLogger(EjemploSpringJUnitApplication.class);
12
13     public Cliente save(Cliente cliente) {
14         if(cliente==null) {
15             logger.trace("Cliente Service: Error al guardar el cliente");
16         }
17         return cliente;
18     }
19
20     public List<Cliente> findAll() {
21         Cliente cliente1 = new Cliente("Cliente1","apellido1","fono1","direccion1","email1");
22         Cliente cliente2 = new Cliente("Cliente2","apellido2","fono2","direccion2","email2");
23         List<Cliente> lista = Arrays.asList(cliente1,cliente2);
24         return lista;
25     }
26
27     public String eliminarPorId(Long id) {
28         if(id!=null) {
29             logger.trace("Cliente Service: cliente eliminado");
30         }
31         return "Eliminación realizada correctamente";
32     }
33
34     public Cliente buscarId(Long id) {
35         Cliente cliente3 = new Cliente("Cliente3","apellido3","fono3","direccion3","email3");
36         return cliente3;
37     }
38 }
39
40
41
42
43
44
45
46 }
```

Imagen 11. La clase Service.

Fuente: Desafío Latam

```
1 package (ejemploSpring/src/main/java/com/ejemplo/ es;  
2 repositories/ClienteRepository.java  
3 import org.springframework.data.jpa.repository.JpaRepository;..  
7  
8 @Repository  
9 public interface ClienteRepository extends JpaRepository<Cliente, Long>{  
10  
11 }  
12
```

Imagen 12. La clase Repository.
Fuente: Desafío Latam

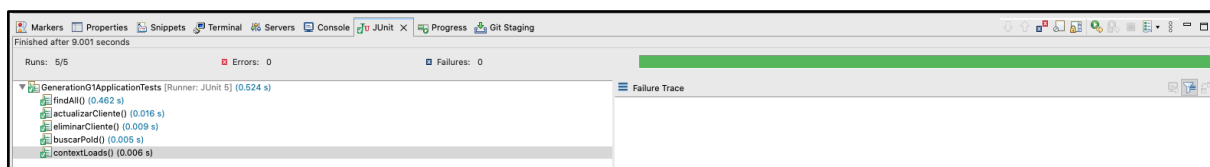


Imagen 13. Test ejecutados.
Fuente: Desafío Latam

Preguntas de proceso

Reflexiona:

- Respecto a los ejercicios desarrollados, ¿hay algo que me haya resultado difícil de desarrollar o entender?
- ¿Cuáles son los pasos para realizar la creación de un proyecto Spring Boot?
- ¿Cómo se crea la clase servicio?
- ¿Cómo se crean los test y se ejecutan?
- Desde Eclipse, ¿dónde puedo crear un proyecto Spring?
- ¿Cuál es el proceso para ejecutar los test desde una aplicación Spring Boot Java?
- Según tu conocimiento, ¿Qué es un @Service?
- Según tu conocimiento, ¿Qué es @Test?



Referencias bibliográficas

- <https://www.java.com/es/>
- <https://dev.java/>